

Computer Organization & Architecture

Unit-10

Computer Arithmetic

Sweetu Sureja
Assistant Professor
IT Department
V.V.P. Engineering College, Rajkot

Outline

- Fixed Point Numbers:
 - I. Addition and Subtraction
 - II. Multiplication Algorithm
 - III. Division Algorithm
- Floating Point Numbers:
 - I. Addition and Subtraction
 - II. Multiplication Algorithm
 - III. Division Algorithm

Fixed Point Numbers

Addition and Subtraction

Addition and Subtraction

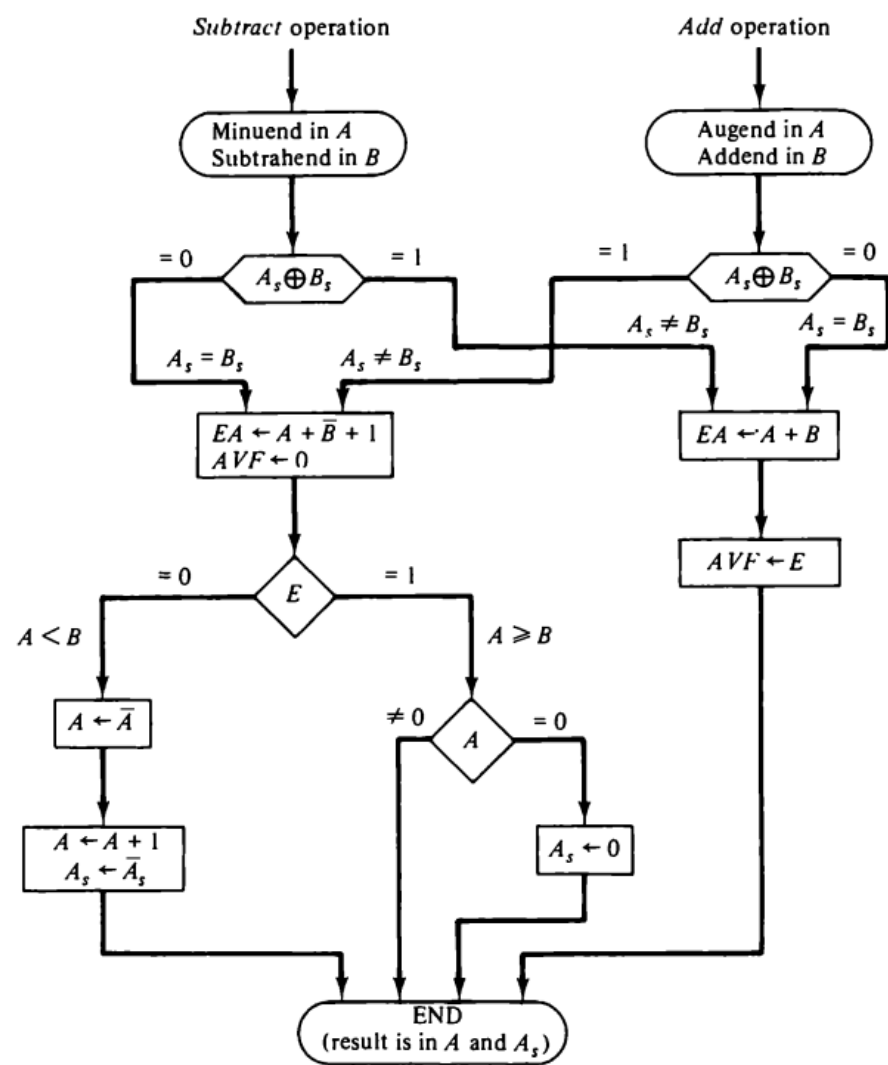
Same sign → Add, Different sign → Subtract, Bigger wins

TABLE 10-1 Addition and Subtraction of Signed-Magnitude Numbers

Operation	Add Magnitudes	Subtract Magnitudes		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$			
$(+A) + (-B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (+B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A + B)$			
$(+A) - (+B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$			
$(-A) - (+B)$	$-(A + B)$			
$(-A) - (-B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$

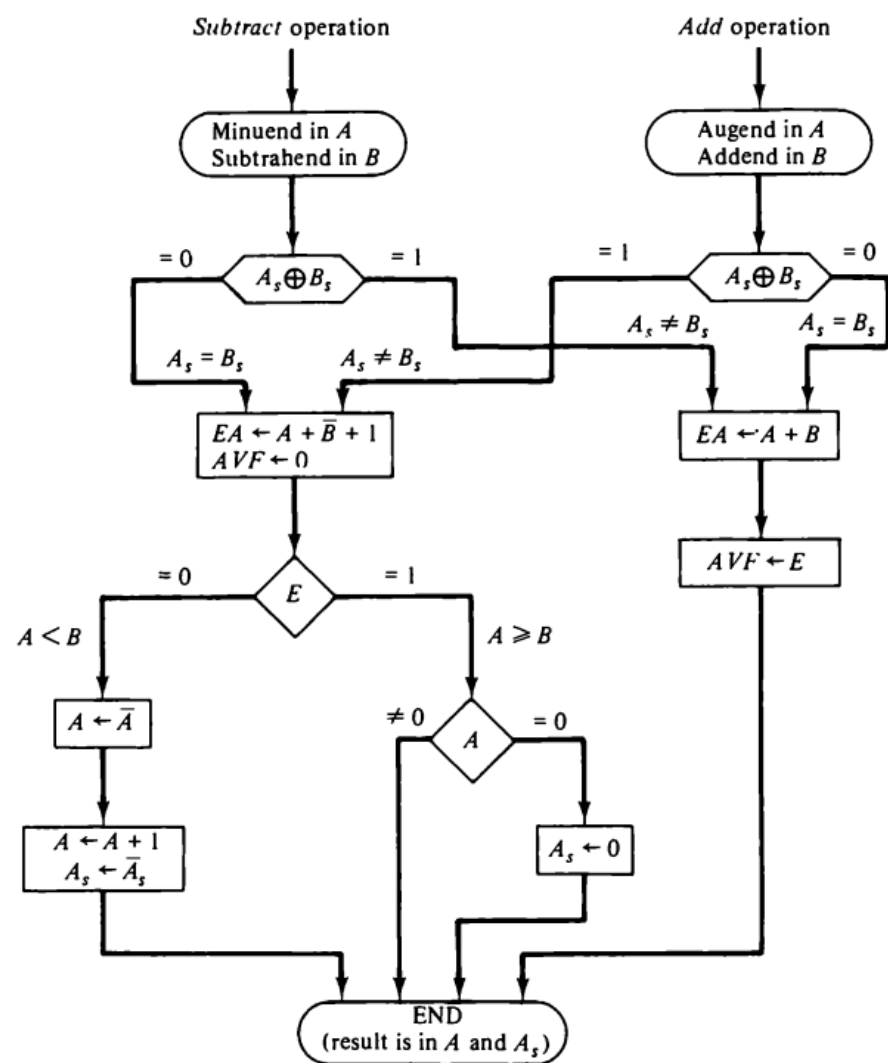
Addition and Subtraction (Signed Magnitude numbers)

- The two signs A_s and B_s are compared by an EX-OR gate.
- If the output of the gate is 0 the signs are identical; If it is 1, the signs are different.
- For an add operation, identical signs dictate that the magnitudes be added. For a subtract operation, different signs dictate that the magnitudes be added.
- The magnitudes are added with a microoperation $EA \leftarrow A + B$, where EA is a register that combines E and A . The carry in E after the addition constitutes an overflow if it is equal to 1. The value of E is transferred into the add-overflow flip-flop AVF .



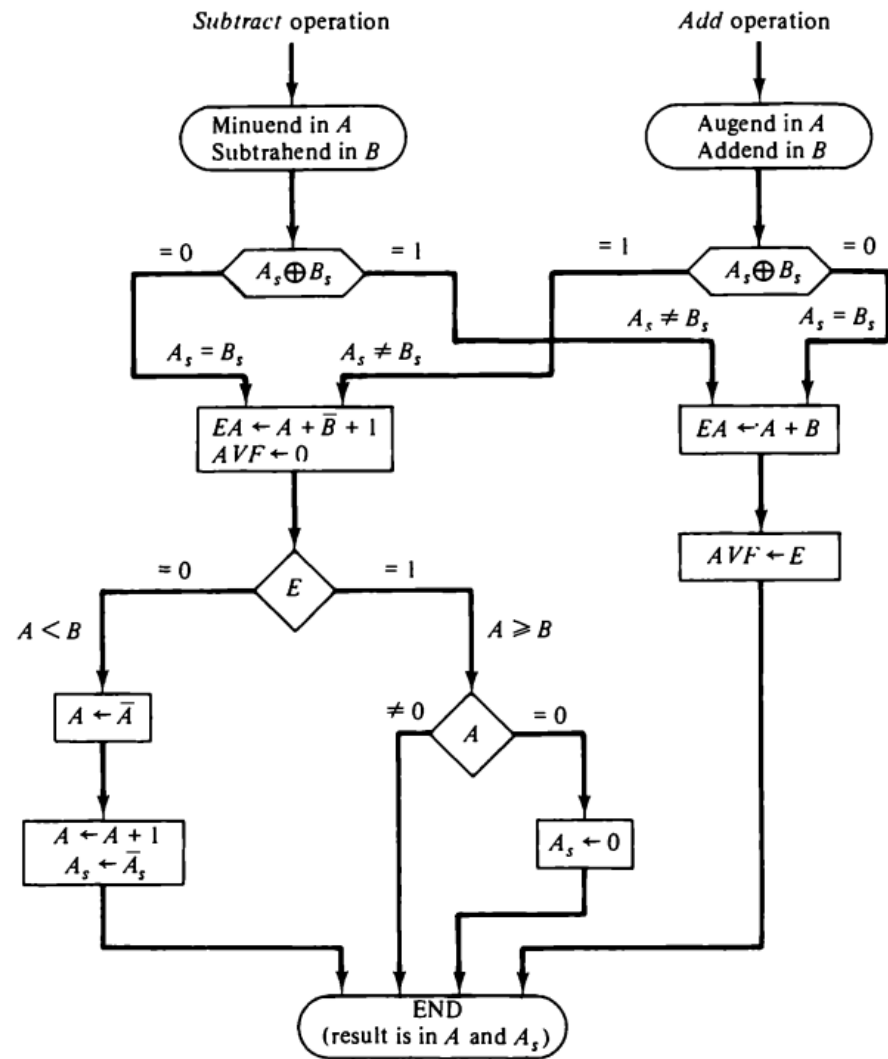
Addition and Subtraction (Signed Magnitude numbers)

- The two magnitudes are subtracted if the signs are different for an add operation or identical for a subtract operation.
- The magnitudes are subtracted by adding A to the 2's complemented B. No overflow can occur if the numbers are subtracted so AVF is cleared to 0.
- 1 in E indicates that $A \geq B$ and the number in A is the correct result. If this number is zero, the sign A must be made positive to avoid a negative zero.
- 0 in E indicates that $A < B$. For this case it is necessary to take the 2's complement of the value in A. The operation can be done with one microoperation $A \leftarrow -A' + 1$.



Addition and Subtraction (Signed Magnitude numbers)

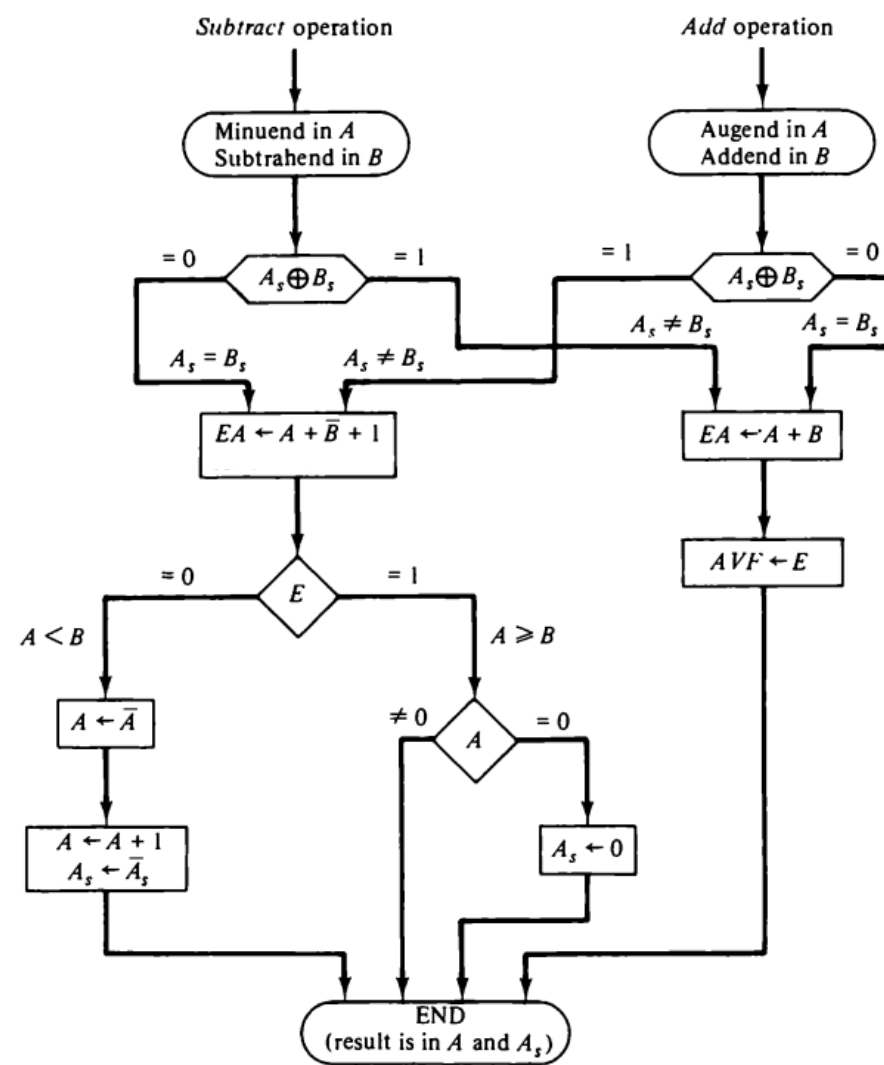
- However, we assume that the A register has circuits for microoperations complement and increment, so the 2's complement is obtained from these two microoperations.
- In other paths of the flowchart, the sign of the result is the same as the sign of A, so no change in A is required. However, when $A < B$, the sign of the result is the complement of the original sign of A. It is then necessary to complement A, to obtain the correct sign.
- The final result is found in register A and its sign in A_s . The value in AVF provides an overflow indication. The final value of E is immaterial.



Addition and Subtraction (Signed Magnitude numbers)

❖ Example:

- $A=(-5)$ and $B=(+6)$ Perform Addition
- Here $A=0101$ and $B=0110$
- $A_s=1$ and $B_s=0$
- Now do $A_s \oplus B_s$ then A_s and B_s are not same so that go through the subtraction part of flowchart.
- Now $EA = A + \bar{B} + 1$
- $EA = 0101 + 1001 + 1$
- $A = 1111$ and $E=0$
- $E=0$ so that $A < B$
- $A = \bar{A} + 1 = 0000 + 1 = 0001$ and $A_s = \bar{A}_s = 0$
- So Result is $A=0001$ and $A_s=0$ i.e. $(+1)$ in decimal number system.



Addition and Subtraction (Signed 2's complement numbers)

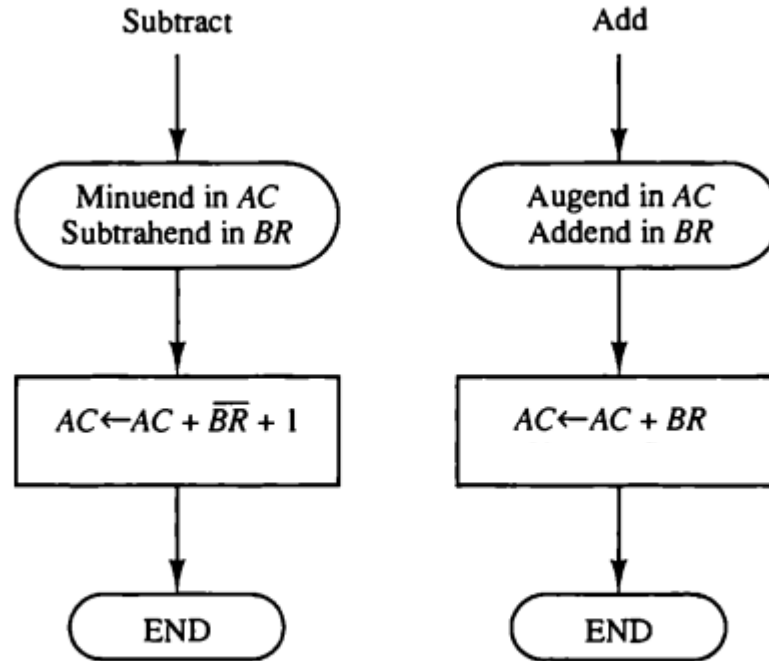


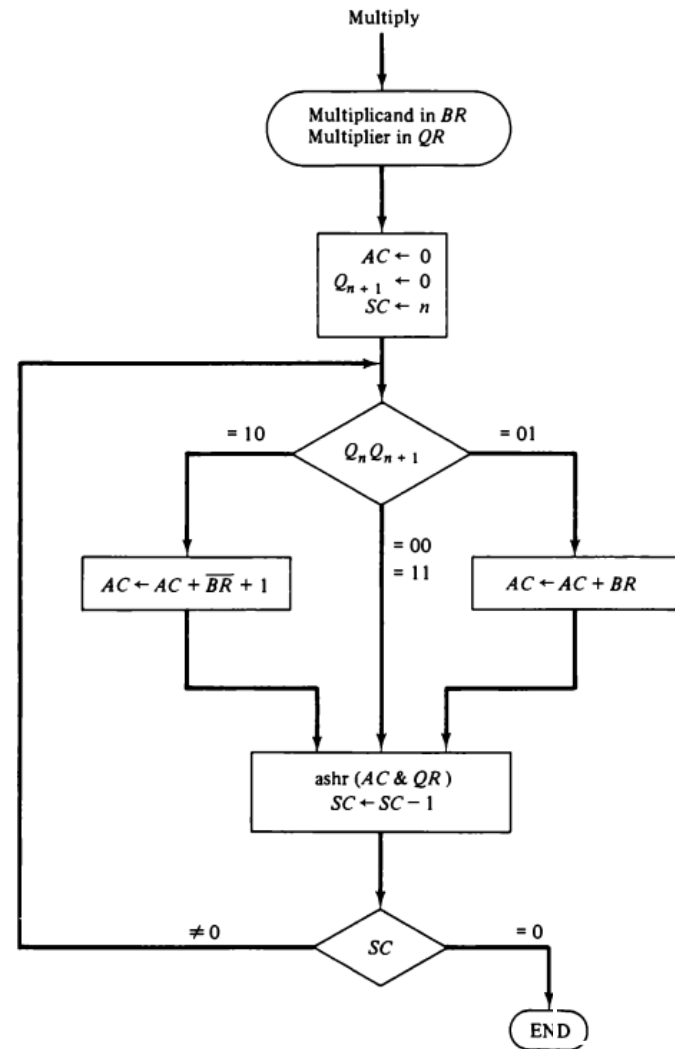
Figure 10-4 Algorithm for adding and subtracting numbers in signed-2's complement representation.

Multiplication Algorithm

Booth Multiplication Algorithm

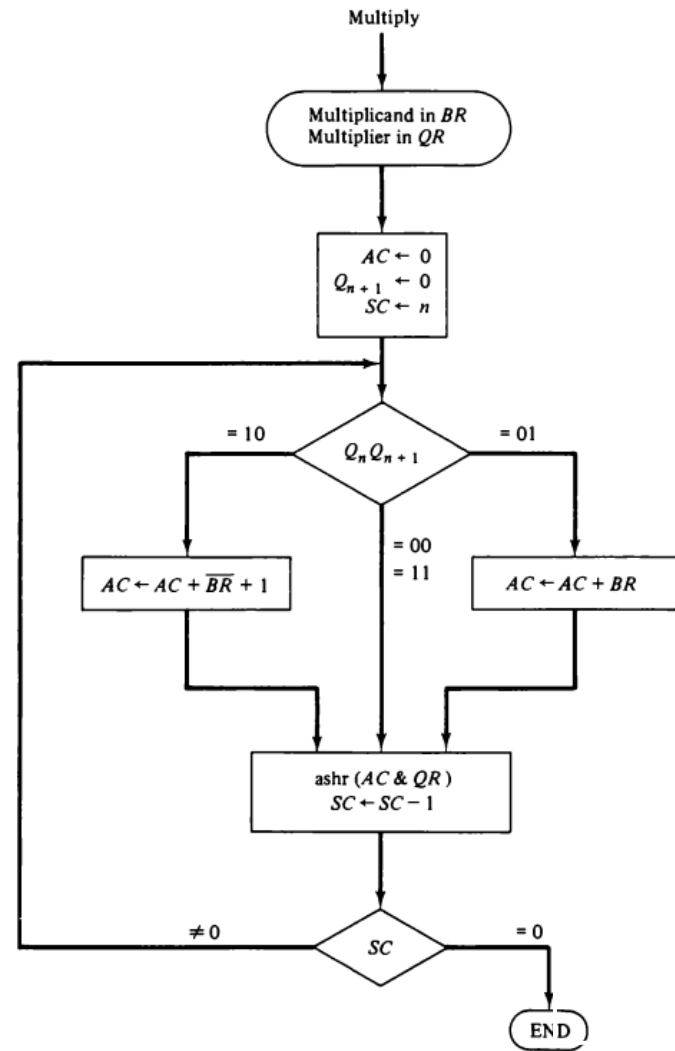
(signed-2's complement representation)

- Booth algorithm gives a procedure for multiplying binary integers in signed-2's complement representation.
- Multiplicand is stored in BR and Multiplier is stored in QR.
- Initially $AC=0$ (no. of bits in QR), $Q_{n+1}=0$ and $SC=no. of bits in multiplier$.**
- Q_n is the LSB bit of QR.**
- Check for values of Q_n and Q_{n+1} .
- If the two bits are equal to 10, this requires a subtraction of the multiplicand from AC.
- If the two bits are equal to 01, this requires the addition of the multiplicand to the AC.



Booth Multiplication Algorithm (signed-2's complement representation)

- When the two bits are equal, it does not require any changes.
- We will perform arithmetic shift right on the values of AC & BR.
- Next decrement the value of SC by 1.
- We will do this process till the value of SC becomes zero.

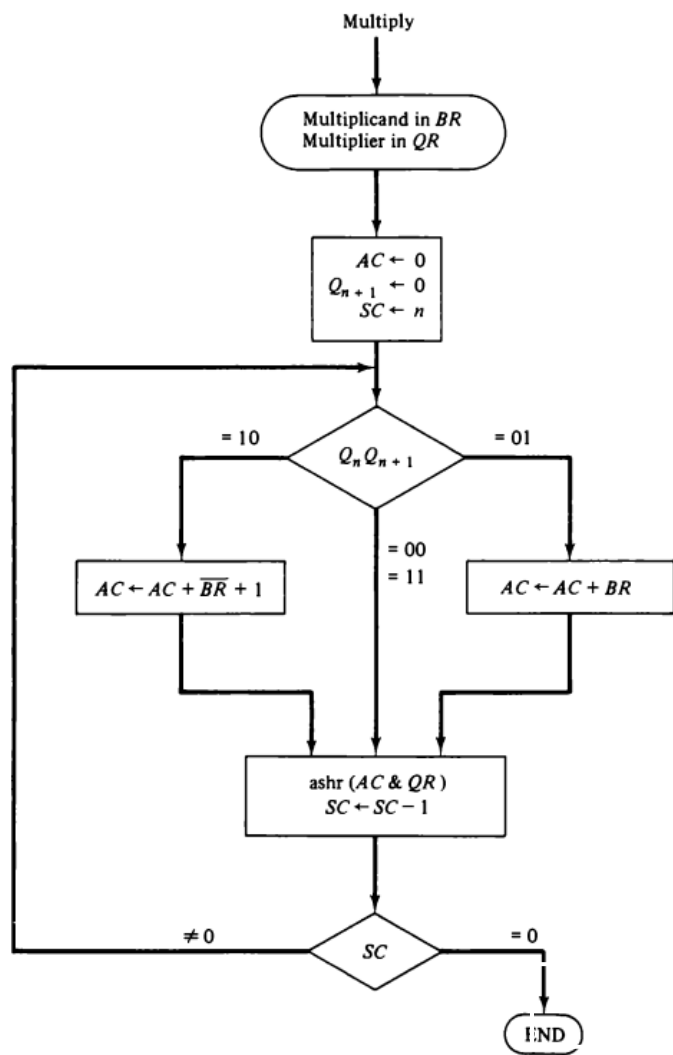


Multiply the multiplier(QR)(-13) with multiplicand(BR)(-9) using booth's algorithm

9: 01001
1's complement of 9: 10110
+ 1
(BR)2's complement of 9: 10111 (-9)

13: 01101
1's complement of 13: 10010
+ 1
(QR)2's complement of 13: 10011 (-13)

$Q_n Q_{n+1}$	$BR = 10111$ $\overline{BR} + 1 = 01001$	AC	QR	Q_{n+1}	SC
	Initial	00000	10011	0	5
1 0	Subtract BR	01001			
		01001			
	ashr	00100	11001	1	4
1 1	ashr	00010	01100	1	3
0 1	Add BR	10111			
		11001			
	ashr	11100	10110	0	2
0 0	ashr	11110	01011	0	1
1 0	Subtract BR	01001			
		00111			
	ashr	00011	10101	1	0



Multiplication Algorithm

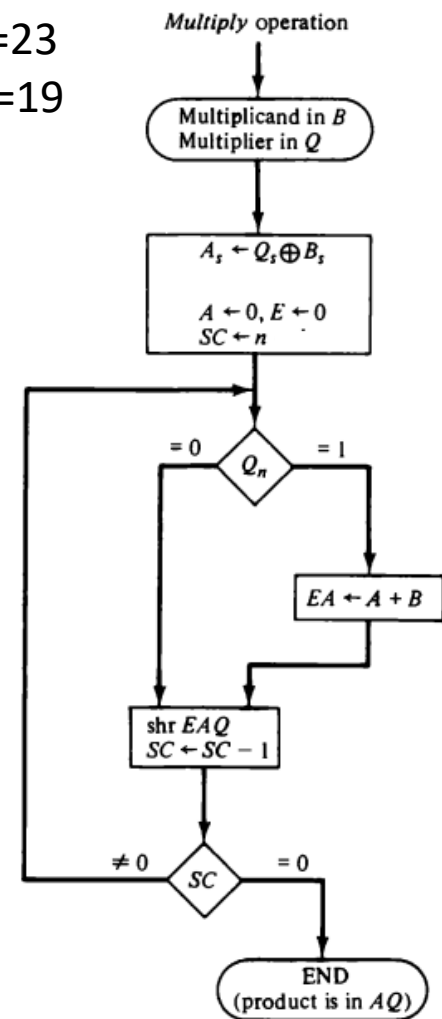
(signed magnitude numbers)

$B=10111=23$

$Q=10011=19$

$B*Q=437$

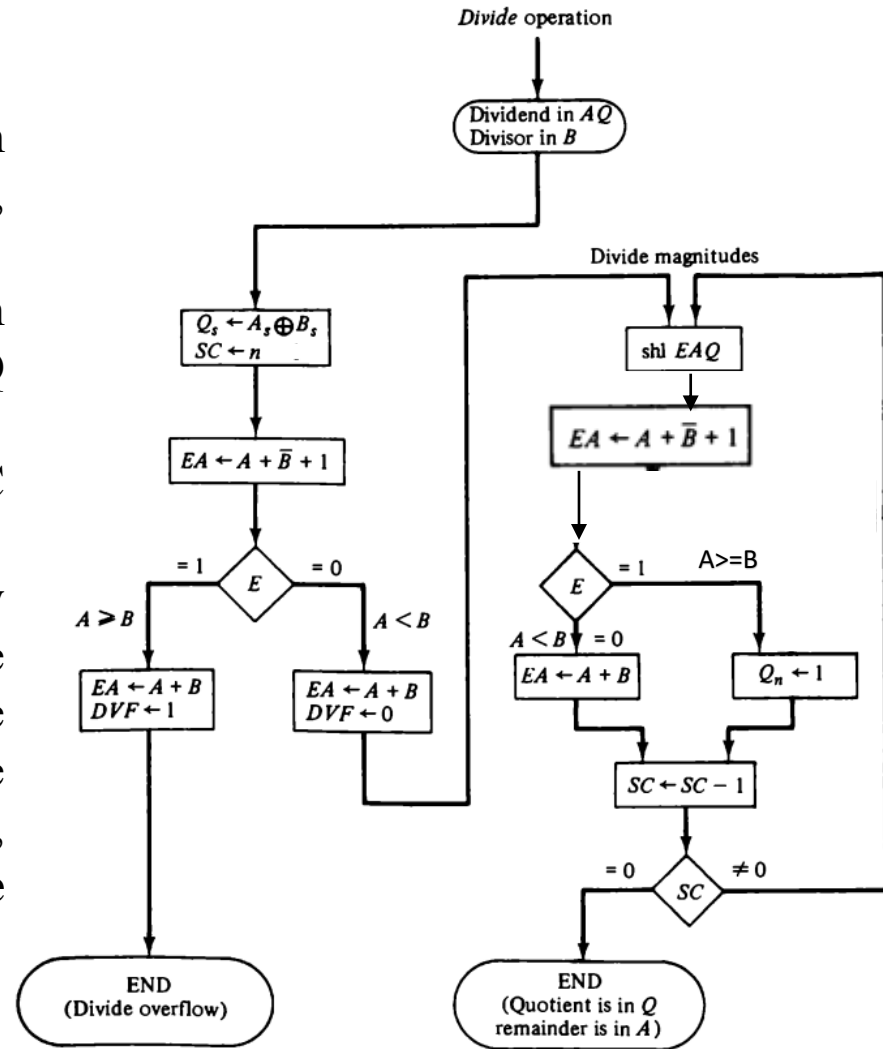
Multiplicand $B = 10111$	E	A	Q	SC
Multiplier in Q	0	00000	10011	5
$Q_n = 1$; add B		<u>10111</u>		
First partial product	0	10111		
Shift right EAQ	0	01011	11001	4
$Q_n = 1$; add B		<u>10111</u>		
Second partial product	1	00010		
Shift right EAQ	0	10001	01100	3
$Q_n = 0$; shift right EAQ	0	01000	10110	2
$Q_n = 0$; shift right EAQ	0	00100	01011	1
$Q_n = 1$; add B		<u>10111</u>		
Fifth partial product	0	11011		
Shift right EAQ	0	01101	10101	0
Final product in $AQ = 0110110101$				



Division Algorithm (Restoring Algorithm)

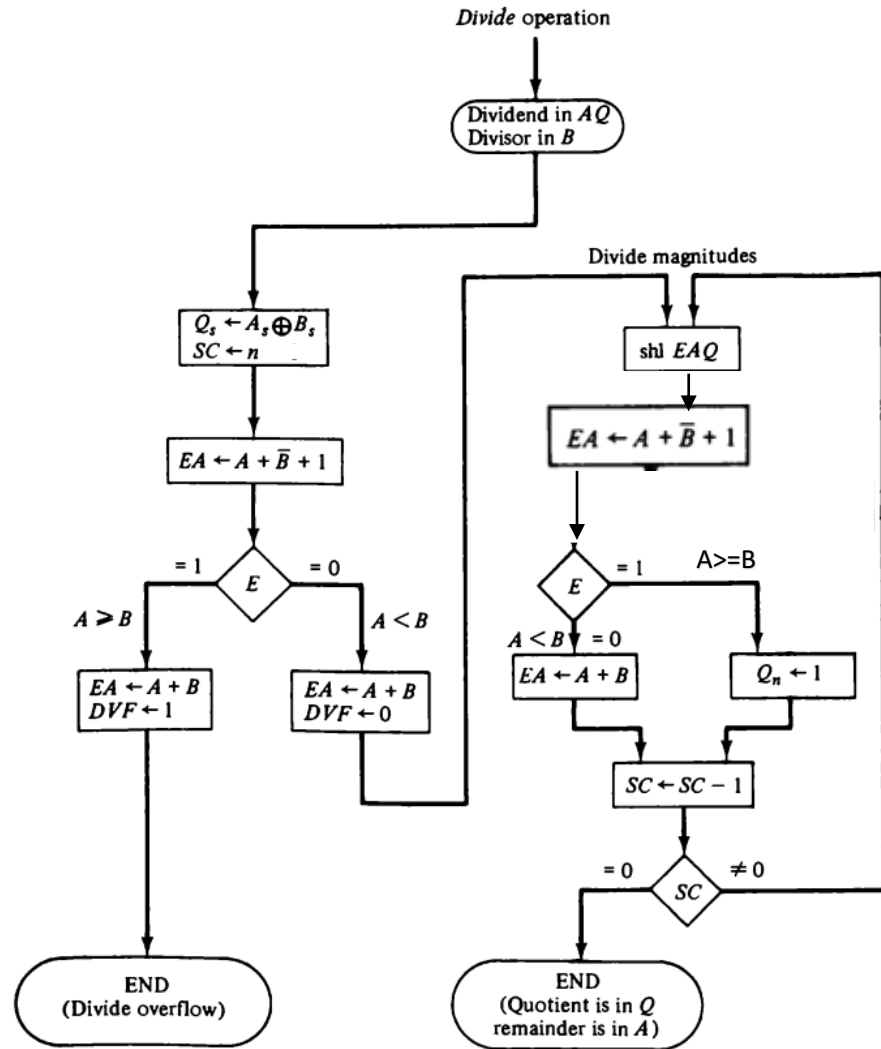
Division Algorithm

- The dividend is in A and Q and the divisor in B. The sign of the result is transferred into Q, to be part of the quotient.
- After the operation Remainder will be stored in A register and Quotient will be stored in Q register.
- A constant is set into the sequence counter SC to specify the number of bits in the quotient.
- A divide-overflow condition is tested by subtracting the divisor in B from half of the bits of the dividend stored in A. If $A \geq B$, the divide-overflow flip-flop DVF is set and the operation is terminated prematurely. If $A < B$, no divide overflow occurs so **the value of the dividend is restored by adding B to A.**

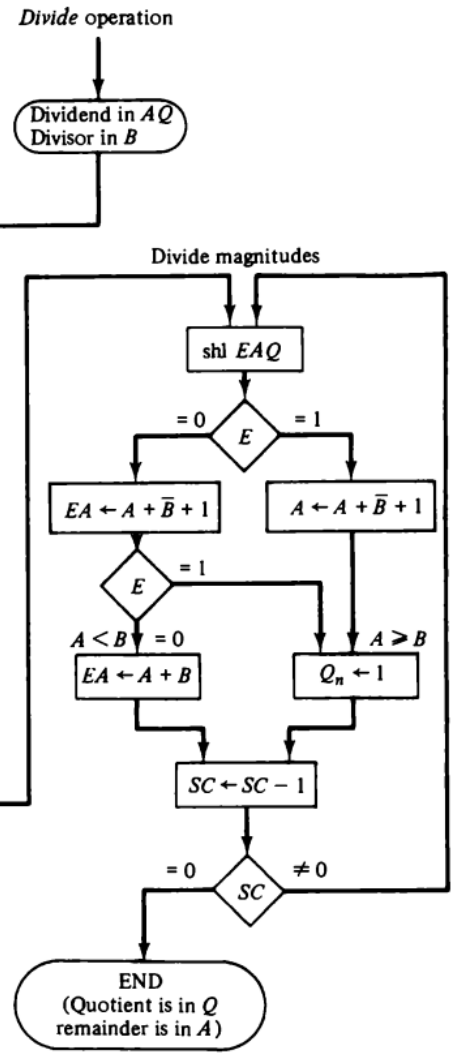


Division Algorithm

- The division of the magnitudes starts by shifting the dividend in A Q to the left with the high-order bit shifted into E.
- Perform subtraction by adding 2's complement value and the carry is transferred into E.
- If $E = 1$, it signifies that $A \geq B$; therefore, Q_n is set to 1. If $E = 0$, it signifies that $A < B$ and the original number is restored by adding B to A . In the latter case we leave a 0 in Q_n (0 was inserted during the shift).
- This process is repeated again with register A holding the partial remainder.
- After SC becomes zero, the quotient magnitude is formed in register Q and the remainder is found in register A.



$A+B+1$
 $=01110+01111$
 $=11101$ and
 $E=0$ So, $A < B$
 Now $EA = A+B$



Divisor $B = 10001$,

$\bar{B} + 1 = 01111$

Dividend:
 shl EAQ
 add $\bar{B} + 1$
 $E = 1$
 Set $Q_n = 1$
 shl EAQ
 Add $\bar{B} + 1$
 $E = 1$
 Set $Q_n = 1$
 shl EAQ
 Add $\bar{B} + 1$
 $E = 0$; leave $Q_n = 0$
 Add B
 Restore remainder
 shl EAQ
 Add $\bar{B} + 1$
 $E = 1$
 Set $Q_n = 1$
 shl EAQ
 Add $\bar{B} + 1$
 $E = 0$; leave $Q_n = 0$
 Add B
 Restore remainder
 Neglect E
 Remainder in A :
 Quotient in Q :

E	A	Q	SC
	01110	00000	5
0	11100	00000	
	01111		
1	01011		
1	01011	00001	4
0	10110	00010	
	01111		
1	00101		
1	00101	00011	3
0	01010	00110	
	01111		
0	11001	00110	
	10001		
1	01010		2
0	10100	01100	
	01111		
1	00011		
1	00011	01101	1
0	00110	11010	
	01111		
0	10101	11010	
	10001		
1	00110	11010	0
	00110		
		11010	

Floating Point Arithmetic Operations

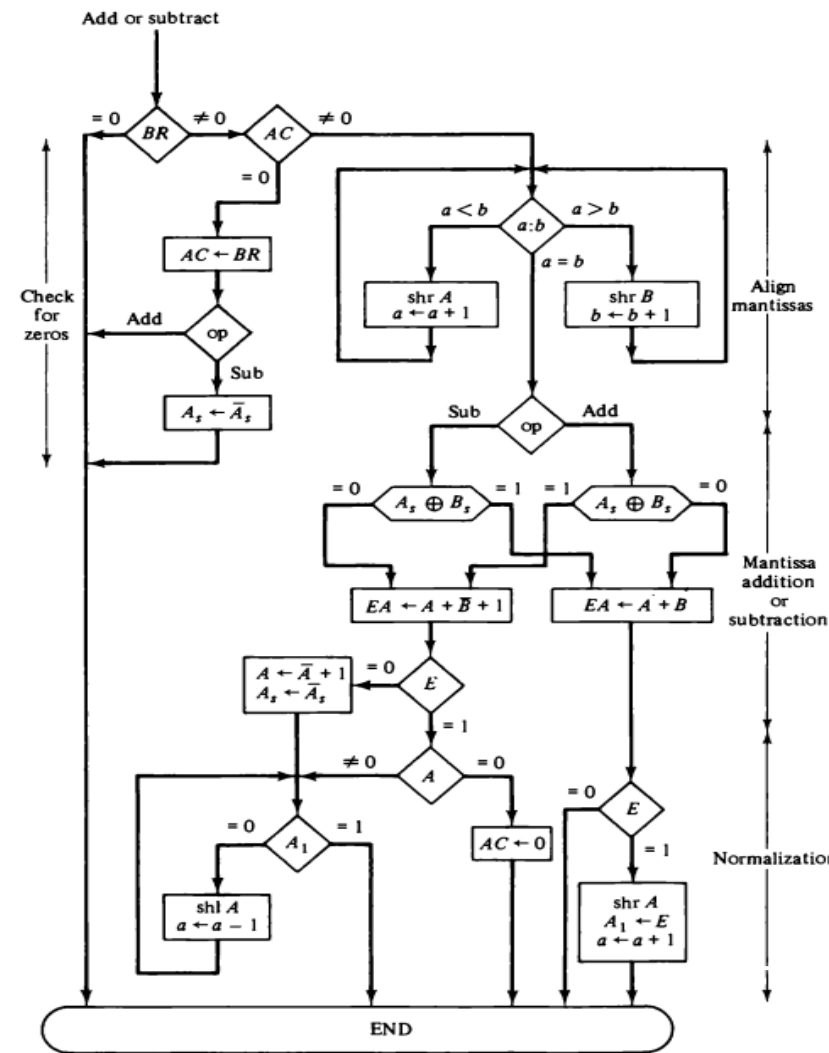
Floating Point Arithmetic Operations

- A floating point number in computer registers consists of **two parts: a mantissa m and an exponent e .**
- The two parts represent a number obtained from multiplying m times a radix r raised to the value of e ; thus
- $$m \times r^e$$
- The mantissa may be a fraction or an integer.
- For example, assume a fraction representation and a radix 10. The decimal number 537.25 is represented in a register with $m = 53725$ and $e = 3$ and is interpreted to represent the floating-point number
- $$.53725 \times 10^3$$
- A floating-point number is **normalized** if the most significant digit of the mantissa is **nonzero**.

Addition and Subtraction

Addition and Subtraction

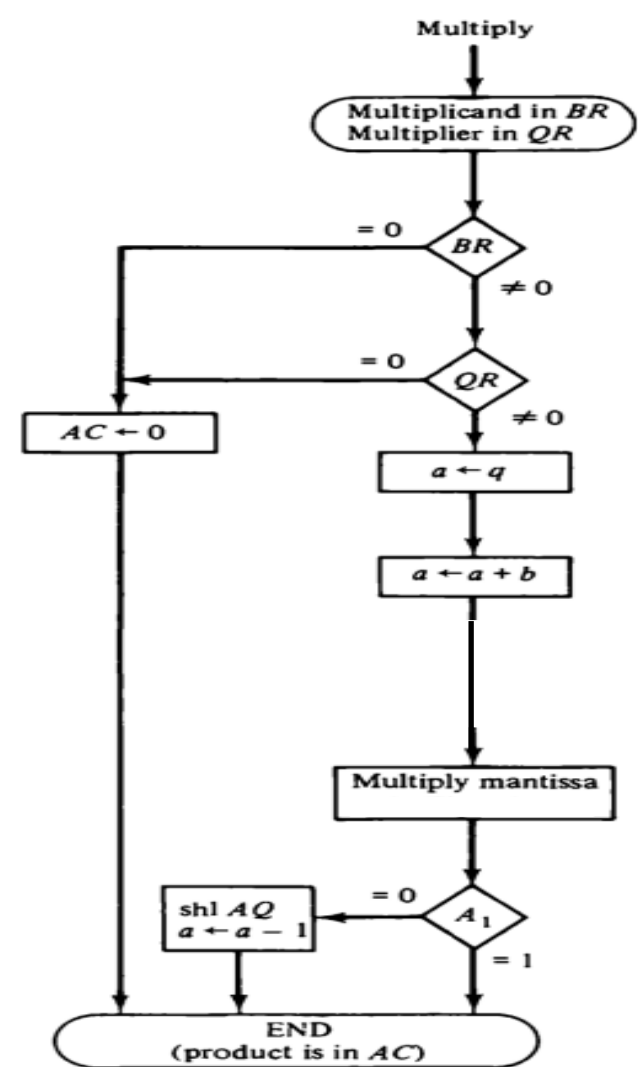
- During addition or subtraction, the two floating-point operands are in AC and BR.
- The sum or difference is formed in the AC. The algorithm can be divided into four consecutive parts:
 1. Check for zeros.
 2. Align the mantissas.
 3. Add or subtract the mantissas.
 4. Normalize the result.
- If BR is equal to zero, the operation is terminated, with the value in the AC being the result.
- If AC is equal to zero, we transfer the content of BR into AC and also complement its sign if the numbers are to be subtracted.
- If neither number is equal to zero, we proceed to align the mantissas.



- The magnitude comparator attached to exponents a and b provides three outputs that indicate their relative magnitude.
- If the two exponents are equal, we go to perform the arithmetic operation. If the exponents are not equal, the mantissa having the smaller exponent is shifted to the right and its exponent incremented.
- This process is repeated until the two exponents are equal.
- The magnitude part is added or subtracted depending on the operation and the signs of the two mantissas.
- If an overflow occurs when the magnitudes are added, it is transferred into flip-flop E. If E is equal to 1, the bit is transferred into A1 and all other bits of A are shifted right.
- The exponent must be incremented to maintain the correct number. No underflow may occur in this case because the original mantissa that was not shifted during the alignment was already in a normalized position.
- If the magnitudes were subtracted, the result may be zero or may have an underflow.
- If the mantissa is zero, the entire floating-point number in the AC is made zero.
- Otherwise, the mantissa must have at least one bit that is equal to 1. The mantissa has an underflow if the most significant bit in position A1 is 0.
- In that case, the mantissa is shifted left and the exponent decremented.
- The bit in A1 is checked again and the process is repeated until it is equal to 1.
- When $A1 = 1$, the mantissa is normalized and the operation is completed.

Multiplication

- The multiplication of two floating-point numbers requires that we multiply the mantissas and add the exponents.
- No comparison of exponents or alignment of mantissas is necessary.
- The multiplication algorithm can be subdivided into four parts:
 1. Check for zeros.
 2. Add the exponents.
 3. Multiply the mantissas.
 4. Normalize the product.
- The two operands are checked to determine if they contain a zero. If either operand is equal to zero, the product in the AC is set to zero and the operation is terminated.

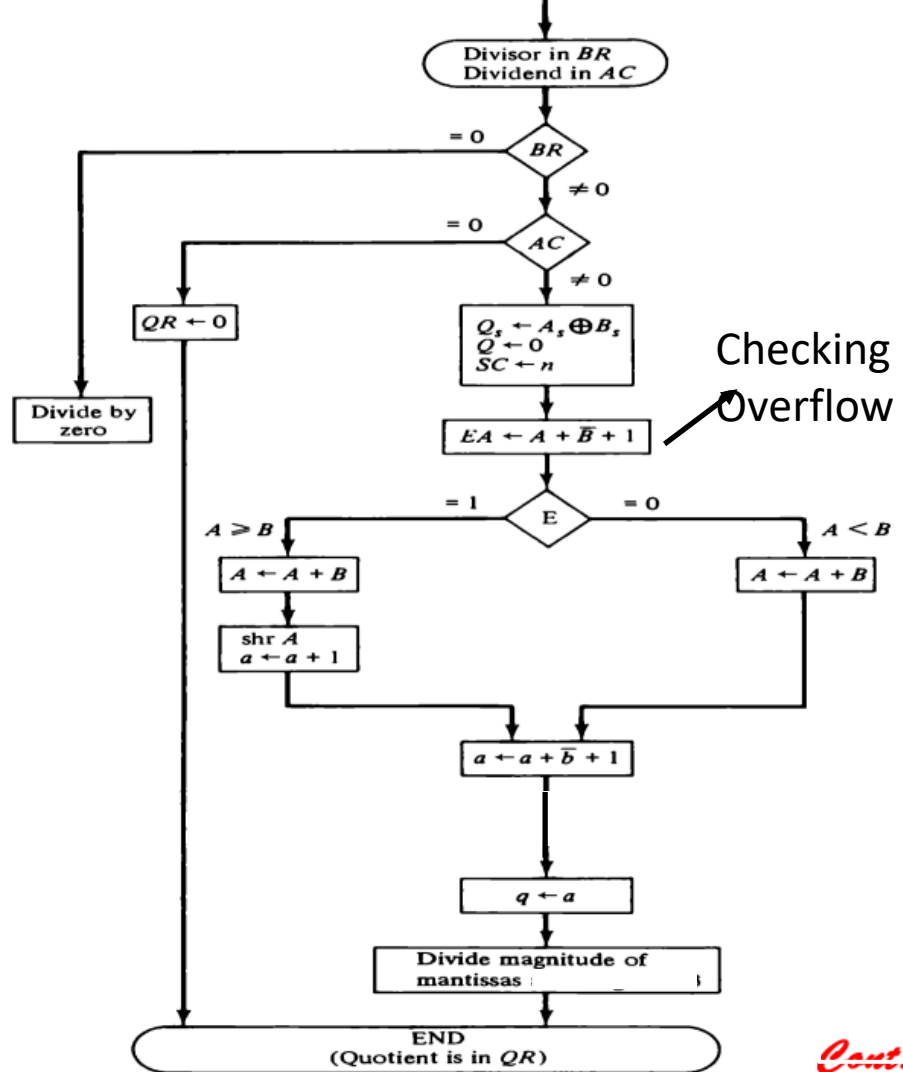


Multiplication

- If neither of the operands is equal to zero, the process continues with the exponent addition.
- The exponent of the multiplier is in q and the adder is between exponents a and b .
- It is necessary to transfer the exponents from q to a , add the two exponents, and transfer the sum into a . Since both exponents are biased by the addition of a constant, the exponent sum will have double this bias.
- The multiplication of the mantissas is done as in the fixed-point case with the product residing in A and Q .
- Overflow cannot occur during multiplication, so there is no need to check for it.
- The product may have an underflow, so the most significant bit in A is checked. If it is a 1, the product is already normalized. If it is a 0, the mantissa in AQ is shifted left and the exponent decremented.
- The multiplier and multiplicand were originally normalized and contained fractions.
- Although the low-order half of the mantissa is in Q , we do not use it for the floating-point product. Only the value in the AC is taken as the product.

Division

- Floating-point division requires that the exponents be subtracted and the mantissas divided.
- The division algorithm can be subdivided into five parts:
 - Check for zeros.
 - Initialize registers and evaluate the sign.
 - Align the dividend (Check for Overflow).
 - Subtract the exponents.
 - Divide the mantissas.
- The two operands are checked for zero. If the divisor is zero, it indicates an attempt to divide by zero, which is an illegal operation.



- The operation is terminated with an error message. An alternative procedure would be to set the quotient in QR to the most positive number possible (if the dividend is positive) or to the most negative possible (if the dividend is negative).
- If the dividend in AC is zero, the quotient in QR is made zero and the operation terminates.
- If the operands are not zero, we proceed to determine the sign of the quotient and store it in Qs.
- The sign of the dividend in A, is left unchanged to be the sign of the remainder.
- The Q register is cleared and the sequence counter SC is set to a number equal to the number of bits in the quotient.
- The proper alignment requires that the fraction dividend be smaller than the divisor.
- The two fractions are compared by a subtraction test. The carry in E determines their relative magnitude.
- The dividend fraction is restored to its original value by adding the divisor. If $A \geq B$, it is necessary to shift A once to the right and increment the dividend exponent.
- Since both operands are normalized, this alignment ensures that $A < B$. Next, the divisor exponent is subtracted from the dividend exponent.

- Since both exponents were originally biased, the subtraction operation gives the difference without the bias.
- The bias is then added and the result transferred into q because the quotient is formed in QR .
- The magnitudes of the mantissas are divided as in the fixed-point case.
- After the operation, the mantissa quotient resides in Q and the remainder in A . The floating-point quotient is already normalized and resides in QR.

GTU Questions

Q.1 Explain booth's multiplication algorithm with example.

Q.2 Show the contents of registers E, AC, BR, QR and SC during the process of multiplying 11111 with 10101.

Q.3 Assume $A = +6$ and $B = +7$, apply Booth algorithm for multiplying A and B. Make necessary assumptions if required.

Q.4 Draw neat and clean flowchart for divide operation. Explain with example.

Q.5 Assume a computer system uses 5 bit (1 sign + 4 Magnitude) registers and 2's complement representation. Perform multiplication of number 10 with the smallest number in this system using booth algorithm. Show step-by-step multiplication process.

Q.6 Explain using flowchart the working of Booth's multiplication algorithm of signed-2's complement numbers.

Q.7 Explain addition and subtraction with signed magnitude data. Also list hardware required for the same.

Q.2 Show the contents of registers E, AC, BR, QR and SC during the process of multiplying 11111 with 10101.

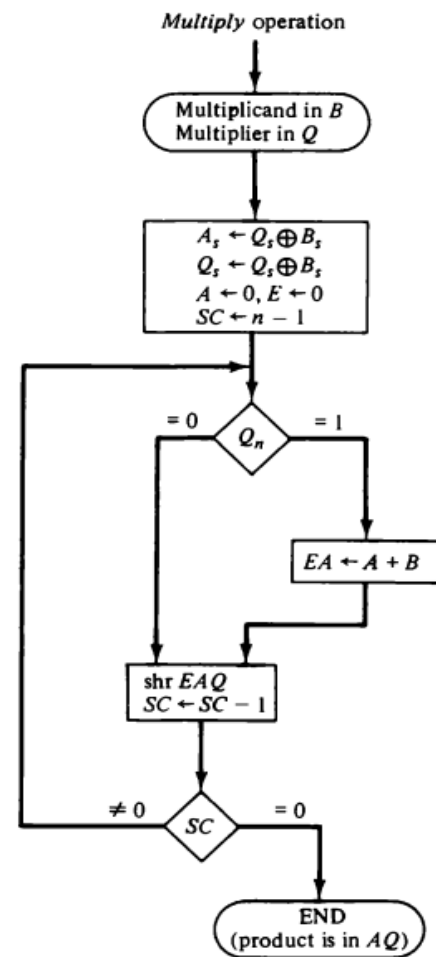
Initially $E=0, AC=00000, BR=11111, QR=10101, SC=5$

$BR=1111=(31)_{10}$

$QR=10101=(21)_{10}$

Final result is in AC & QR = $1010001011 = (651)_{10}$

Operation	E	AC	QR	SC
Initial	0	00000	10101	5
EA=A+B		11111		
	0	11111	10101	
SHR EAQ	0	01111	11010	4
SHR EAQ	0	00111	11101	3
EA=A+B		11111		
	1	00110	11101	
SHR EAQ	0	10011	01110	2
SHQ EAQ	0	01001	10111	1
EA=A+B		11111		
	1	01000	10111	
SHR EAQ	0	10100	01011	0



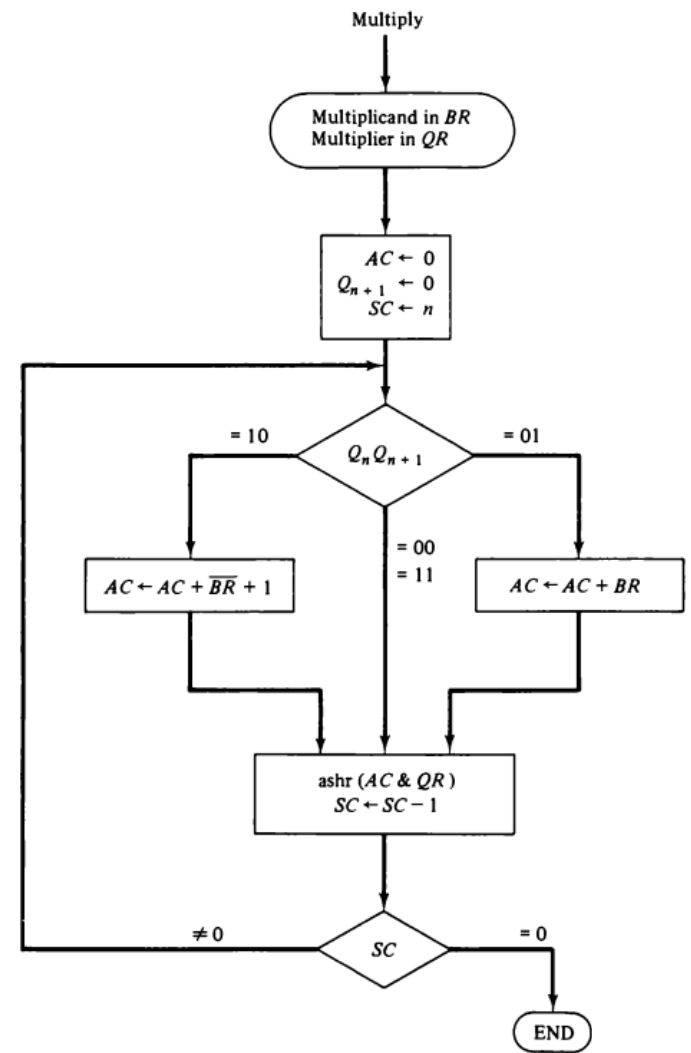
Q.3 Assume $A = +6$ and $B = +7$, apply Booth algorithm for multiplying A and B. Make necessary assumptions if required.

Multiplicand= $A=+6$ BR=00110 $\overline{BR+1}=11001+1=11010$

Multiplier= $B=+7$ QR=00111

Answer is in AC and QR = 00001 01010 = 42

$Q_n Q_{n+1}$	Operation	AC	QR	Q_{n+1}	SC
	Initial	00000	00111	0	5
10	Subtract BR	11010			
		11010	00111		
	Ashr	11101	00011	1	4
11	Ashr	11110	10001	1	3
11	Ashr	11111	01000	1	2
01	Add BR	00110			
		00101	01000		
	Ashr	00010	10100	0	1
00	Ashr	00001	01010	0	0



Q.5 Assume a computer system uses 5 bit (1 sign + 4 Magnitude) registers and 2's complement representation. Perform multiplication of number 10 with the smallest number in this system using booth algorithm. Show step-by-step multiplication process.

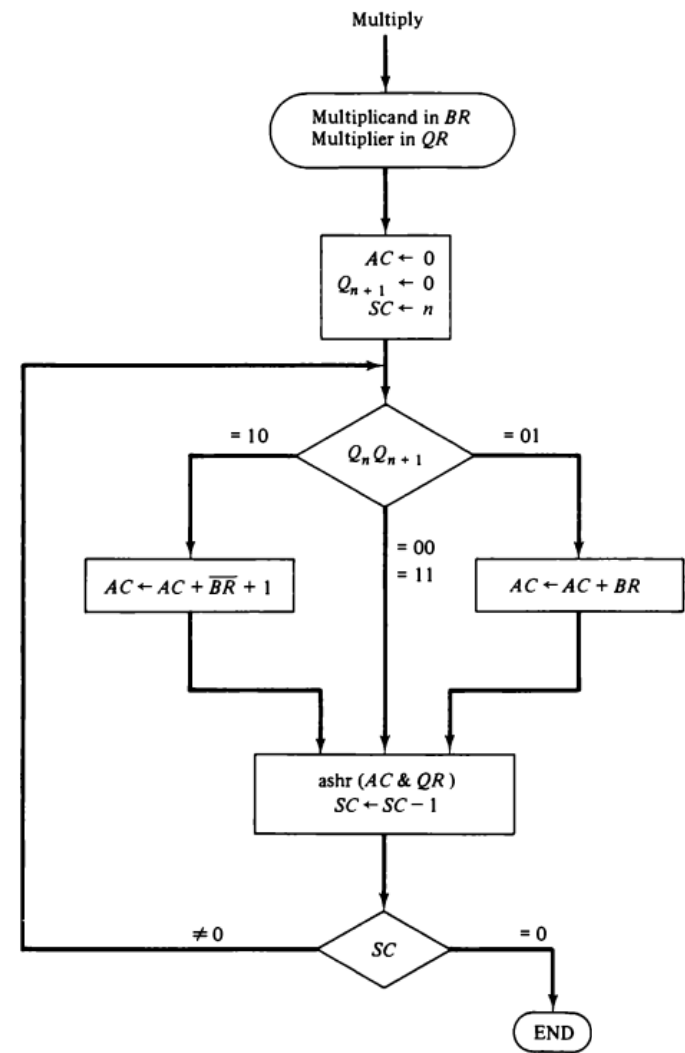
Multiplicand=M=10 BR=01010 BR+1=10101+1=10110

Multiplier=Q=-16 QR=10000

Answer is in AC and QR = 11011 00000

Now answer is negative so find the 2's complement of the answer= 00101 00000(-160)

$Q_n Q_{n+1}$	Operation	AC	QR	Q_{n+1}	SC
	Initial	00000	10000	0	5
00	Ashr	00000	01000	0	4
00	Ashr	00000	00100	0	3
00	Ashr	00000	00010	0	2
00	Ashr	00000	00001	0	1
10	Subtract BR	10110			
		10110	00001		
	Ashr	11011	00000	1	0



Q.7 Explain addition and subtraction with signed magnitude data. Also list hardware required for the same.

List of hardware:

1. Sign Bit Comparator: Compare the sign of numbers to determine whether to perform addition or subtraction.
2. Binary Adder: perform addition
3. Binary Subtractor: perform subtraction when signs are different
4. Sign Control Logic: decides sign of the result
5. Overflow Detector: check the overflow conditions

Thanking You